

BRAGBOARD PROJECT WEEK 6 to 8 REPORT:-

NAME : CHITIRALA GIRIDHAR

GMAIL: GIRIDHARCHITIRALA@GMAIL.COM

GITHUB LINK:- <https://github.com/springboardmentorx5768/Project-2.git>

deployment link :- <https://giridhar-bragboard.vercel.app>

here using loveble & vercel app to deploy project...

Executive summary

This document explains the design, implementation, and testing of **ShoutBoard** — an internal appreciation tool that allows employees to post shout-outs, react, and comment, with admin moderation and analytics. The report covers Milestones 1–4 (Weeks 1–8), provides succinct code examples for key features, lists common issues encountered during development, and describes rectification steps and testing strategies. This is written as a continuous narrative (no boxes) with clear headings for easy reading.

Table of contents

1. Project overview and goals
2. System architecture
3. Week-by-week development summary
4. Key components and short code samples
 - Frontend (React)
 - Backend (FastAPI)
 - Database schema
 - Real-time updates (WebSocket/Socket.IO)
5. Commenting, avatars, timestamps, nesting
6. Reactions and live updates
7. Admin dashboard, moderation, reporting
8. Export, deployment, CI/CD, testing & QA
9. Common issues & rectifications
10. Conclusion and next step

1. Project overview and goals

ShoutBoard is a web application where employees can post appreciation messages (shout-outs), add reactions, comment, and tag colleagues. Admins can moderate content, view analytics, and export reports. Goals include:

- Rapid MVP within 8 weeks
 - Clean UX for posting and viewing shout-outs
 - Real-time reactions and comments
 - Admin moderation and exportable analytics
-

2. System architecture (high level)

- **Frontend:** React (functional components + hooks), optional UI library (Tailwind/CSS), client-side routing for pages.
- **Backend:** FastAPI for REST endpoints, WebSocket endpoints for live updates.
- **Database:** PostgreSQL (primary), Redis for caching and pub/sub for real-time events (optional).
- **Auth:** JWT-based auth for API calls; session cookies optional for browser UX.
- **Hosting:** Containerized (Docker), deployable to AWS/GCP/Heroku/Vercel.

Data flow: user -> frontend -> REST API -> DB; for live events: backend publishes to Redis pub/sub or uses WebSocket connections directly.

3. Week-by-week development summary

Week 1 (Milestone 1 start)

- Project scaffold: React app + FastAPI backend
- Basic user model and authentication endpoints
- Basic UI for posting shout-outs

Week 2

- Post listing, filtering (by department, sender, date)
- Attach images support (optional)
-

Week 3 (Milestone 2 start)

- Reactions: like, clap, celebrate; backend endpoints
- Real-time updates using WebSocket or Socket.IO
- Show reaction buttons with live counts

Week 4

- Commenting system under shout-outs
- Avatars, timestamps; optional nested comments
- Moderation basics: soft-delete flags

Week 5 (Milestone 3 start)

- Admin tools: view reports, top contributors
- Ability to delete posts/comments

Week 6

- Moderation UI: reported content review
- Live updates for reporting and resolving

Week 7 (Milestone 4 start)

- Export reports (CSV/PDF)
- Leaderboard and gamification

Week 8

- Deployment, final testing, polish, responsive styling
 - QA checklist and bug-fixes
-

4. Key components & short code samples

Below are compact code snippets to illustrate implementation approaches. These are *short* and focus only on essentials.

4.1 Frontend — React (short)

Posting a shout-out (React + fetch):

```
// PostShoutout.js (simplified)

import React, {useState} from 'react';

export default function PostShoutout({token, onPosted}){

  const [text,setText] = useState("");

  async function submit(){

    const res = await fetch('/api/shoutouts',{

      method:'POST',

      headers:{'Content-Type':'application/json','Authorization':'Bearer '+token},

      body:JSON.stringify({text})

    });

    if(res.ok){ setText(""); onPosted(); }

  }

  return (

    <div>

      <textarea value={text} onChange={e=>setText(e.target.value)} />

      <button onClick={submit}>Shout</button>

    </div>

  );

}
```

Reaction button (optimistic update):

```
async function toggleReaction(shoutId, reactionType){

  // update UI first (optimistic)

  setLocalCount(prev=>prev+1);

}
```

```

const res = await fetch(`/api/shoutouts/${shoutId}/reactions`, {
  method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer '+token },
  body: JSON.stringify({ type: reactionType })
});

if (!res.ok) { /* rollback UI update */ }
}

```

Listening for live updates (WebSocket):

```

useEffect(() => {
  const ws = new WebSocket('wss://yourdomain/ws');
  ws.onmessage = e => { const msg = JSON.parse(e.data); handleEvent(msg); };
  return () => ws.close();
}, []);

```

4.2 Backend — FastAPI (short)

Minimal shoutout model (Pydantic + SQLAlchemy):

```

# models.py

from sqlalchemy import Column, Integer, String, ForeignKey, DateTime, Boolean
from sqlalchemy.sql import func

class Shoutout(Base):
    __tablename__ = 'shoutouts'

    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey('users.id'))
    text = Column(String)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    is_deleted = Column(Boolean, default=False)

```

Create shoutout endpoint:

```

@app.post('/api/shoutouts')
def create_shoutout(payload: ShoutoutCreate, user=Depends(get_current_user)):

```

```
s = Shoutout(user_id=user.id, text=payload.text)
```

```
db.add(s); db.commit(); db.refresh(s)
```

```
# publish to websocket or redis pubsub
```

```
return s
```

Reaction endpoint:

```
@app.post('/api/shoutouts/{id}/reactions')
```

```
def add_reaction(id:int, payload:ReactionCreate, user=Depends(get_current_user)):
```

```
    # upsert reaction record for (user,shout, type)
```

```
    # update aggregated counts table or cache
```

```
    return {'status':'ok'}
```

4.3 Database schema (short)

Tables: users, shoutouts, reactions, comments, reports, tags, teams.

Key relationships:

- shoutouts.user_id -> users.id
- reactions.shoutout_id -> shoutouts.id (unique per user+type)
- comments.parent_id -> comments.id for nesting

Indexes: created_at indexes, user_id, index on (shoutout_id, type) for reaction counts.

5. Commenting system details

- **Features:** comments under shout-outs, avatars, timestamps, optional nesting (1–2 levels recommended for clarity).
- **Frontend:** lazy-load comments and show "View N comments" to reduce initial payload.
- **Backend:** comment model with parent_id for nesting and is_deleted for soft-deletes.
- **Moderation:** reported flag on comment; admins can resolve or permanently delete.

Short comment model (SQLAlchemy):

```
class Comment(Base):
```

```
id = Column(Integer, primary_key=True)
shoutout_id = Column(Integer, ForeignKey('shoutouts.id'))
user_id = Column(Integer, ForeignKey('users.id'))
parent_id = Column(Integer, ForeignKey('comments.id'), nullable=True)
text = Column(String)
created_at = Column(DateTime, server_default=func.now())
is_deleted = Column(Boolean, default=False)
```

Nesting UX: show up to 2 levels inline. Deeper threads can be rendered in a dedicated comment modal.

6. Reactions with live updates

- Use an aggregated counter table or cache to avoid heavy joins on read.
- When a reaction is added/removed:
 1. Upsert the reactions table for that user + type.
 2. Update aggregate counts (increment/decrement) in shoutouts or a dedicated reaction_counts table.
 3. Publish an event over WebSocket/Redis to notify connected clients.

Potential optimizations:

- Debounce frequent toggles from a user.
 - Store ephemeral counts in Redis for extremely high traffic and periodically flush to DB.
-

7. Admin dashboard & moderation

Admin features:

- View top contributors (by shoutouts, reactions received)
- Most-tagged employees
- Report management: view, resolve, delete
- Delete posts/comments (soft delete + audit log)
- Export CSV/PDF reports

Short route example (FastAPI):

```
@app.get('/api/admin/top-contributors')
```

```
def top_contributors(...):
```

```
    # SQL: select user_id, count(*) as total from shoutouts group by user_id order by total
    desc limit 20
```

```
    pass
```

Audit & logs: keep an actions table that records admin actions (who, what, when, reason) for compliance.

8. Export, deployment, CI/CD, testing & QA

Exporting reports: use Python libraries (pandas to create CSV, reportlab or WeasyPrint for PDF). For large exports, run an async background job and provide a download link.

Deployment checklist:

- Dockerize backend and frontend
- Use environment variables for secrets
- Run migrations (Alembic)
- Set up managed DB (RDS/Cloud SQL)
- Configure HTTPS and CORS

CI/CD: run tests (unit, integration), linting, automated builds. Use GitHub Actions or GitLab CI.

Testing & QA:

- Unit tests for backend logic (auth, reactions, moderation)
 - Integration tests for API endpoints
 - Manual UI tests (posting, comment nesting, real-time behavior)
 - Accessibility check (keyboard nav, alt text)
-

9. Common issues encountered & rectification steps

1. Race conditions on reactions (counts mismatch):

- *Cause:* concurrent updates to counts without atomicity.

- *Fix:* Use DB transactions and SELECT FOR UPDATE on aggregate rows or maintain counts in Redis with atomic INCR/DECR, then persist.

2. **Comment nesting performance:**

- *Cause:* retrieving entire deep comment trees in a single query.
- *Fix:* limit nesting depth, lazy-load deeper threads, use recursive CTEs carefully, cache popular threads.

3. **Real-time scaling issues:**

- *Cause:* single WebSocket server can't handle many connections.
- *Fix:* use a pub/sub layer (Redis) and horizontally scale WebSocket servers behind a load balancer; or adopt a managed realtime service.

4. **Image attachment storage & serving:**

- *Cause:* storing images in DB or local disk.
- *Fix:* store in S3-compatible object storage, serve via CDN, validate file types and sizes.

5. **Moderation bypass / abusive content:**

- *Cause:* users posting offensive content.
- *Fix:* add reporting, automated profanity filter (basic rules), require admin review for flagged posts, rate-limits, and user suspension flow.

6. **Auth token leakage in browser logs:**

- *Cause:* storing raw JWT in localStorage and exposing in full to client errors.
- *Fix:* use httpOnly cookies for tokens or encrypt storage; ensure logs scrub sensitive headers.

7. **Export timeouts for large reports:**

- *Cause:* synchronous export generation.
- *Fix:* background job queue (Celery/RQ), notify user when file is ready, chunked CSV generation.

8. **Inconsistent timestamps across timezones:**

- *Cause:* saving naive datetimes or inconsistent timezone handling.
- *Fix:* store UTC in DB, render localized times on client using Intl APIs; include timezone metadata.

Each issue above includes detection tips (logs to check) and suggested tests to confirm the fix.

10. Conclusion & next steps

The MVP described here is achievable in eight weeks with a small team (1–2 frontend, 1 backend, 1 QA). Prioritize core features (post, react, comment, moderate) and iterate on analytics and gamification. Use monitoring (Sentry/Prometheus) and real-user testing to guide further refinements.

Appendix A — Short code snippets (extras)

Comment creation (FastAPI):

```
@app.post('/api/shoutouts/{id}/comments')

def add_comment(id:int, payload:CommentCreate, user=Depends(get_current_user)):

    c = Comment(shoutout_id=id, user_id=user.id, text=payload.text,
parent_id=payload.parent_id)

    db.add(c); db.commit(); db.refresh(c)

    publish_ws_event({'type':'comment_created','data':serialize(c)})

    return c
```

Export CSV (Python/pandas):

```
import pandas as pd

rows = db.execute("select * from shoutouts").fetchall()

df = pd.DataFrame(rows)

df.to_csv('/tmp/shoutouts_export.csv', index=False)
```

Dockerfile (FastAPI minimal):

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .
```

```
CMD ["uvicorn","app.main:app","--host","0.0.0.0","--port","80"]
```

Appendix B — QA checklist (brief)

- End-to-end: user can post, see it listed, react, comment, and admin can moderate.
 - Security: verify auth, role-based access, sanitize HTML inputs.
 - Performance: absolute page load < 2s for feed; reaction latency < 1s for websocket path.
 - Accessibility: ARIA labels for buttons, keyboard nav.
-

Appendix C — Estimated file contents to include when handing off

- README with setup steps
- API spec (OpenAPI) and Postman collection
- DB migrations (Alembic)
- Docker compose for local dev
- Unit and integration test suites